

CSED211: Malloc lab Report

컴퓨터공학과 20240940 장해영

1. Overview

해당 과제는 malloc과 관련된 기본함수를 사용하지 않고 동적할당을 구현하고, 최적화하는 과제이다. 우선 교과서에 있는 코드를 이용하여 기본 코드를 짜고, 이를 최적화하는 방식으로 과제를 수행하였다.

```
Results for mm malloc:
trace valid util ops secs Kops
0 yes 99% 5854 0.014248 480
1 yes 99% 4805 0.015759 385
2 yes 55% 12000 0.117788 102
3 yes 55% 8000 0.112888 71
4 yes 51% 24000 0.411621 58
5 yes 51% 16000 0.392273 41
6 yes 99% 5848 0.006694 874
7 yes 99% 5032 0.006674 754
8 yes 66% 14400 0.00091157722
9 yes 66% 14400 0.00090160000
10 yes 99% 6648 0.011427 582
11 yes 99% 5683 0.013953 467
12 yes 100% 5380 0.016477 327
13 yes 100% 4537 0.014367 316
14 yes 92% 4800 0.011659 412
15 yes 92% 4800 0.013555 354
16 yes 92% 4800 0.012317 390
17 yes 92% 4800 0.006100 787
18 no - - - -
19 no - - - -
20 no - - - -
21 no - - - -
22 yes 66% 12 0.000000 40000
23 yes 66% 12 0.000000 60000
24 yes 89% 12 0.000000 40000
25 yes 89% 12 0.000000 60000
Total - - - -
Terminated with 4 errors
[hjyang@programming2 malloclab-handout]$
```

교과서에 있는 코드와 realloc은 원래 있던 코드를 이용하여 코드를 실행해 보니 다음과 같은 결과를 얻을 수 있었다. 중간에 오류가 발생한 trace의 오류 메시지를 확인했을 때 mm_realloc과 관련된 함수임을 확인하였고, 이를 해결하기 위해 코드를 수정해야한다는 것을 확인하였다.

2. 초기설정

교과서를 참고하여 매크로와 전역변수, 함수를 구성하였다.

각 매크로의 의미는 다음과 같다.

```
// macro
#define WSIZE 4
#define DSIZE 8
#define CHUNKSIZE (1<<12)

#define MAX(x, y) ((x) > (y) ? (x) : (y))

#define PACK(size, alloc) ((size) | (alloc))

#define GET(p) (*(unsigned int *) (p))
#define PUT(p, val) (*(unsigned int *) (p) = (val))

#define GET_SIZE(p) (GET(p) & ~0x7)
#define GET_ALLOC(p) (GET(p) & 0x1)

#define HDRP(bp) ((char *) (bp) - WSIZE)
#define FTRP(bp) ((char *) (bp) + GET_SIZE(HDRP(bp)) - DSIZE)

#define NEXT_BLKP(bp) ((char *) (bp) + GET_SIZE((char *) (bp) - WSIZE))
#define PREV_BLKP(bp) ((char *) (bp) - GET_SIZE((char *) (bp) - DSIZE))

// global variable
static char* heap_listp;
static char* last_bp;

// additional functions
static void *extend_heap(size_t words);
static void *coalesce(void *bp);
static void *find_fit(size_t asize);
static void place(void *bp, size_t asize);
//int mm_check(void);
```

WSIZE: word size, DSIZE: double word size,

CHUNKSIZE: heap을 확장할 때 default size

MAX: max값을 반환, PACK: 헤더, 푸터의 최종값

GET: 포인터의 주소에 저장된 값 반환

PUT: 포인터의 주소에 값을 저장

GET_SIZE: size정보 반환, GET_ALLOC: 할당여부 반환

HDRP: 헤더포인터 반환, FTRP: 푸터 포인터 반환

NEXT_BLKP: 다음블록, PREV_BLKP: 이전블록 포인터

Global variable은 다음과 같다.

heap_listp: heap의 첫 블록, last_hp: next fit에서 탐색이 끝난 위치를 저장한다.

추가로 정의한 함수는 아래 주어진 함수의 동작을 돕기 위한 helper function이다.

3. mm_init

```
/*
 * mm_init - initialize the malloc package.
 */
int mm_init(void)
{
    // create the initial empty heap
    if ((heap_listp = mem_sbrk(4*WSIZE)) == (void *)-1)
        return -1;
    PUT(heap_listp, 0);
    PUT(heap_listp + (1*WSIZE), PACK(DSIZE, 1));
    PUT(heap_listp + (2*WSIZE), PACK(DSIZE, 1));
    PUT(heap_listp + (3*WSIZE), PACK(0, 1));
    heap_listp += (2*WSIZE);

    // extend the empty heap with a free block of CHUNKSIZE bytes
    if (extend_heap(CHUNKSIZE/WSIZE) == NULL)
        return -1;

    last_bp = (char*)heap_listp;
    return 0;
}
```

교과서의 코드를 참고하여 작성하였다. heap확장에 성공하면 0을, 실패하면 1을 반환한다. 추후 next fit을 적용하기 위해 last_bp를 heap의 첫 블록인 heap_listp로 초기화하는 작업을 해주었다.

4. mm_malloc

```
/*
 * mm_malloc - Allocate a block by incrementing the brk pointer.
 * Always allocate a block whose size is a multiple of the alignment.
 */
void *mm_malloc(size_t size)
{
    size_t asize;
    size_t extendsize;
    char *bp;

    // ignore spurious requests
    if(size == 0) return NULL;

    // adjust block size to include overhead and alignment reqs
    if(size <= DSIZE)
        asize = 2*DSIZE;
    else
        asize = DSIZE * ((size + (DSIZE) + (DSIZE-1)) / DSIZE);

    // search the free list for a fit
    if ((bp = find_fit(asize)) != NULL) {
        place(bp, asize);
        //mm_check();
        return bp;
    }

    // no fit found get more memory and place the block
    extendsize = MAX(asize, CHUNKSIZE);
    if ((bp = extend_heap(extendsize/WSIZE)) == NULL)
        return NULL;
    place(bp, asize);
    //mm_check();
    return bp;
}
```

교과서의 코드를 참고하여 작성하였다. 주어진 size만큼 allocation을 진행하는 함수이다. 기본적으로 implicit free list으로 구현하였다. 할당에 성공한 경우 할당된 포인터를, 할당에 실패한 경우 NULL을 반환한다. 이때 요청한 크기가 DSIZE 즉, 8바이트 이하라면, 최소 크기 블록만큼 할당을 진행하고, 아니라면, 헤더와 푸터의 크기인 DSIZE와 할당을 진행할 크기를 더하고, 8의 정수가 되도록 올림하여 할당해주었다. 다음으로, find_fit함수를 호

출하여 값을 저장할 위치를 찾고, place함수를 이용하여 allocate상태로 전환한다. 만약, 할당할 공간이 없다면 heap을 확장하여 allocate상태로 전환해준다.

5. mm_free

```
/*
 * mm_free - Freeing a block does nothing.
 */
void mm_free(void *ptr)
{
    size_t size = GET_SIZE(HDRP(ptr));

    PUT(HDRP(ptr), PACK(size, 0));
    PUT(FTRP(ptr), PACK(size, 0));
    coalesce(ptr);
    //mm_check();
}
```

교과서의 코드를 참고하여 작성하였다. 헤더와 푸터의 allocate여부를 0으로 설정해준뒤, free블록들의 병합을 진행하기 위해 coalesce함수를 호출한다.

6. mm_realloc

```
/*
 * mm_realloc - Implemented simply in terms of mm_malloc and mm_free
 */
void *mm_realloc(void *ptr, size_t size)
{
    void *oldptr = ptr;
    void *newptr;
    void *nextptr = NEXT_BLKPTR(oldptr);
    size_t copySize;
    size_t oldSize = GET_SIZE(HDRP(oldptr));
    size_t newSize = ALIGN(size + DSIZ);
    size_t combinedSize = oldSize;

    // exption
    if(ptr == NULL){
        return mm_malloc(size);
    }
    if(size == 0){
        mm_free(ptr);
        return NULL;
    }
}
```

```
// have enough space
if(oldSize >= newSize) return oldptr;

// check possible extend
while ((GET_SIZE(HDRP(nextptr)) > 0 && !GET_ALLOC(HDRP(nextptr)))){
    combinedSize += GET_SIZE(HDRP(nextptr));

    if(combinedSize >= newSize){
        PUT(HDRP(oldptr), PACK(combinedSize, 1));
        PUT(FTRP(oldptr), PACK(combinedSize, 1));
        last_bp = oldptr;
        return oldptr;
    }
    nextptr = NEXT_BLKPTR(nextptr);
}

// cannot extend
newptr = mm_malloc(size);
if(newptr == NULL) return NULL;

copySize = oldSize - DSIZ;
memcpy(newptr, oldptr, copySize);
mm_free(oldptr);
//mm_check();
return newptr;
}
```

Realloc의 경우 교과서의 코드를 그대로 사용하지 않고 최적화를 진행하였다. 우선 예외처리를 진행해주었다. 기존 블록이 NULL인 경우, realloc을 진행하지 않아도 되므로 바로 mm_malloc을 호출하였다. 사이즈가 0인 경우, 메모리를 해제하는 것이므로 NULL을 반환하였다.

이제 실제 realloc을 해야하는 경우이다. 세가지 경우로 나눌 수 있다. 첫째, 충분한 공간이 있어 그대로 할당해도 되는 경우 그대로 oldptr을 반환한다. 이제 다음으로 다음 블록을 확인하여 확장이 가능한 지 확인한다. 다음블록이 존재하고, 할당이 가능하다면, 블록끼리 합쳐 사이즈를 늘려준다. 이렇게 크기를 합쳐가며 충분한 공간이 확보되면 병합을 수행하고, 블록 포인터를 반환한다. 마지막으로, 확장이 더 이상불가능하여 copy를 이용하여 새롭게 할당해야 하는 경우, 원래 적혀있던 코드대로 수행해준다.

```
Results for mm malloc:
trace valid util ops secs Kops
0 yes 99% 5694 0.007280 782
1 yes 99% 4805 0.007310 657
2 yes 55% 12000 0.096438 124
3 yes 55% 8000 0.109965 73
4 yes 51% 24000 0.440990 54
5 yes 51% 16000 0.360927 44
6 yes 99% 5848 0.006680 875
7 yes 99% 5032 0.006668 755
8 yes 66% 14400 0.000095152220
9 yes 66% 14400 0.000090160000
10 yes 99% 6648 0.020436 325
11 yes 99% 5683 0.018719 304
12 yes 100% 5380 0.009603 560
13 yes 100% 4537 0.012196 372
14 yes 92% 4800 0.006491 740
15 yes 92% 4800 0.007789 616
16 yes 92% 4800 0.008648 555
17 yes 92% 4800 0.010828 443
18 yes 49% 14401 0.000189 76035
19 yes 49% 14401 0.000185 77843
20 yes 46% 14401 0.000073196735
21 yes 46% 14401 0.000073196467
22 yes 66% 12 0.000000 40000
23 yes 66% 12 0.000000 60000
24 yes 89% 12 0.000000 40000
25 yes 89% 12 0.000000 40000
Total 77% 209279 1.131674 185

Perf index = 46 (util) + 12 (thru) = 59/100
[hyjang@programming2 mallocclab-handout]$
```

이렇게 경우를 나누어 realloc을 수행함으로써 copy를 이용하여 할당하는 경우가 줄어들기에 throughput점수를 늘릴 수 있을 것이라 예상했다. 이렇게 수정한 뒤 프로그램을 돌렸을 때 다음과 같은 결과를 얻을 수 있었다. 위에서 발생하였던 오류는 없어졌지만 여전히 60점 이하의 낮은 점수임을 확인하고, fit의 방식을 first fit이 아닌, next fit으로 수정해야겠다는 생각을 하고 추후 find_fit함수를 next fit으로 수정하였다.

7. Extend_heap

```
// extend_heap
static void *extend_heap(size_t words) {
    char *bp;
    size_t size;

    // allocate an even number of words to maintain alignment
    size = (words % 2) ? (words+1) * WSIZE : words * WSIZE;
    if((long)(bp = mem_sbrk(size)) == -1)
        return NULL;

    // initialize free block header/footer and the epilogue header
    PUT(HDRP(bp), PACK(size, 0));
    PUT(FTRP(bp), PACK(size, 0));
    PUT(HDRP(NEXT_BLKP(bp)), PACK(0, 1));

    //coalesce if the previous block was free
    return coalesce(bp);
}
```

교과서의 코드를 참고하여 작성하였다. Heap을 확장하는 함수이다. Alignment를 만족할 수 있도록 주어진 words가 홀수 크기라면 1을 더한 상태로 확장을 진행해 주었다. 다음으로 mem_sbrk를 호출하여 heap을 확장해주고, 성공하면 확장된 메모리의 시작주소를, 실패하면 -1을 반환해준다. 마지막으로, 새롭게 할당한 블록이 free된 블록과 연결되어 있다면, 이 둘을 병합할 수 있도록 coalesce함수를 호출한 뒤 반환해주었다.

8. Coalesce

```
// coalesce
static void *coalesce(void *bp){
    size_t prev_alloc = GET_ALLOC(FTRP(PREV_BLKP(bp)));
    size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKP(bp)));
    size_t size = GET_SIZE(HDRP(bp));

    // case1
    if (prev_alloc && next_alloc) {
    }
    // case2
    else if (prev_alloc && !next_alloc) {
        size += GET_SIZE(HDRP(NEXT_BLKP(bp)));
        PUT(HDRP(bp), PACK(size, 0));
        PUT(FTRP(bp), PACK(size, 0));
    }
    // case3
    else if (!prev_alloc && next_alloc) {
        size += GET_SIZE(HDRP(PREV_BLKP(bp)));
        PUT(FTRP(bp), PACK(size, 0));
        PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0));
        bp = PREV_BLKP(bp);
    }
    // case4
    else {
        size += GET_SIZE(HDRP(PREV_BLKP(bp))) + GET_SIZE(FTRP(NEXT_BLKP(bp)));
        PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0));
        PUT(FTRP(NEXT_BLKP(bp)), PACK(size, 0));
        bp = PREV_BLKP(bp);
    }
    last_bp = bp;
    return bp;
}
```

교과서의 코드를 참고하여 작성하였다. 블록 병합은 4가지 case로 나누어 생각할 수 있다. Case1은 현재 블록의 앞과 뒤 블록이 모두 할당되어 병합할 블록이 없는 경우이다. 이 경우, 원래 블록 그대로 반환해주면 된다. Case2는 뒤의 블록이 free여서 병합할 수 있는 경우이다. 뒤 블록과 현재 블록을 병합해주고, 현재 블록을 반환한다. Case3는 앞만 빈 블록인 경우이다. 앞 블록과 현재 블록의 크기를 합쳐주고, 현재 블록을 앞블록으로 바꿔준 뒤 반환한다. 마지막 case4는 앞과 뒤 모두 빈블록이어서 모두 합치는 경우이다. 크기를 앞, 현재, 뒤 블록을 모두 더한 값으로 바꿔주고, 현재 블록을 앞 블록으로 바꿔준 뒤 반환한다. Last_bp = bp; 는 next fit을 보장하기 위해 마지막으로 할당에 관여한 블록을 현재 블록인 bp로 바꿔주는 것이다. 이렇게 모든 작업을 끝마친 뒤 bp를 반환해준다.

9. Find_fit

```
// find_fit
static void *find_fit(size_t asize){
    char *bp;

    // next fit
    for(bp = last_bp; GET_SIZE(HDRP(bp)) > 0; bp = NEXT_BLK(bp)){
        if (!GET_ALLOC(HDRP(bp)) && (asize <= GET_SIZE(HDRP(bp)))){
            last_bp = bp;
            return bp;
        }
    }

    // not possible to find free block after next block
    for (bp = heap_listp; bp < last_bp; bp = NEXT_BLK(bp)){
        if (!GET_ALLOC(HDRP(bp)) && (asize <= GET_SIZE(HDRP(bp)))){
            last_bp = bp;
            return bp;
        }
    }

    // cannot find free block
    return NULL;
}
```

throughput 점수를 늘리기 위해 교과서에 나와있는 first fit이 아닌, next fit을 이용하였다. Last_bp 전역 변수를 추가하여 마지막으로 할당에 관여한 블록을 저장하고, 이 위치부터 탐색을 하도록 하였고, 이 위치 이후에서 찾을 수 없었다면 다시 처음으로 돌아서 first fit을 수행하도록 하였다. 이렇게 수행하니 throughput 점수가 대폭 상승한

것을 확인할 수 있었다. 자세한 내용은 result에 첨부하였다.

10. Place

```
// place
static void place(void *bp, size_t asize){
    size_t csize = GET_SIZE(HDRP(bp));

    if((csize - asize) >= (2*DSIZE)){
        PUT(HDRP(bp), PACK(asize, 1));
        PUT(FTRP(bp), PACK(asize, 1));
        bp = NEXT_BLK(bp);
        PUT(HDRP(bp), PACK(csize-asize, 0));
        PUT(FTRP(bp), PACK(csize-asize, 0));
    }
    else{
        PUT(HDRP(bp), PACK(csize, 1));
        PUT(FTRP(bp), PACK(csize, 1));
    }
}
```

교과서의 코드를 참고하여 작성하였다. place함수는 빈 블록에 요청받은 크기의 메모리를 배치한다. 만약, 남은 공간이 최소공간이상이라면, 적당히 분할하여 fragmentation을 방지할 수 있도록 하였다.

11. mm_check

총 5가지를 확인할 수 있도록 하였다.

```
int mm_check(void){
    void *bp = heap_listp;

    // check prologue block
    if(GET_SIZE(HDRP(heap_listp)) != DSIZE || !GET_ALLOC(HDRP(heap_listp))){
        printf("Error: prologue block error\n");
        return 0;
    }

    // check heap
    for(bp = heap_listp; GET_SIZE(HDRP(bp)) > 0; bp = NEXT_BLK(bp)){
        // check aligned
        if((size_t)bp % 8 != 0){
            printf("Error: %p is not aligned\n", bp);
            return 0;
        }
        // check header and footer
        if(GET_HDRP(bp) != GET_FTRP(bp)){
            printf("Error: header and footer does not match at %p\n", bp);
            return 0;
        }
        // check coalescing
        if(!GET_ALLOC(HDRP(bp)) && !GET_ALLOC(HDRP(NEXT_BLK(bp)))){
            printf("Error: contiguous free blocks escaped coalescing at %p\n", bp);
            return 0;
        }
    }

    // check epilogue
    if(GET_SIZE(HDRP(bp)) != 0 || !GET_ALLOC(HDRP(bp))){
        printf("Error: epilogue block error\n");
        return 0;
    }

    // pass check
    return 1;
}
```

프롤로그 블록 확인

: heap의 제일 첫 부분이 제대로 존재하는 지 확인한다.

정렬확인

: 블록이 8바이트 정렬을 만족하는 지 확인한다.

Header, footer 일치 여부 확인

: header와 footer에 있는 정보들이 일치하는 지 확인한다.

Coalescing 실패 확인

: free블록이 연속에서 두 개가 있다면, 병합이 제대로 일어나지 않은 것이므로 이를 확인한다.

에필로그 블록 확인

: heap의 제일 뒷 부분이 제대로 존재하는 지 확인한다.

12. Result

```
Results for mm malloc:
trace  valid  util    ops      secs  Kops
0      yes   90%   5694  0.004458 1277
1      yes   90%   4805  0.004944  972
2      yes   55%  12000  0.016619  722
3      yes   55%   8000  0.016712  479
4      yes   51%  24000  0.013879 1729
5      yes   51%  16000  0.016110  993
6      yes   93%   5848  0.000999 5854
7      yes   93%   5032  0.002289 2198
8      yes   66%  14400  0.000087165899
9      yes   66%  14400  0.000088163823
10     yes   94%   6648  0.004793 1387
11     yes   94%   5683  0.005066 1122
12     yes   96%   5380  0.004563 1179
13     yes   96%   4537  0.003248 1397
14     yes   89%   4800  0.003920 1224
15     yes   89%   4800  0.003912 1227
16     yes   87%   4800  0.003666 1309
17     yes   87%   4800  0.003673 1307
18     yes   42%  14401  0.000285 50477
19     yes   42%  14401  0.000269 53635
20     yes   53%  14401  0.000086167844
21     yes   53%  14401  0.000075191503
22     yes   66%    12  0.000000 40000
23     yes   66%    12  0.000000 40000
24     yes   89%    12  0.000000 40000
25     yes   89%    12  0.000000 40000
Total      75% 209279  0.109743 1907

Perf index = 45 (util) + 40 (thru) = 85/100
[hyjang@programming2 malloc-lab-handout]$
```

함수 최적화를 완료한 뒤 최종 결과는 다음과 같다. Next fit을 사용한 경우가 first fit을 사용한 경우보다 훨씬 높은 throughput 점수를 얻을 수 있음을 확인하였다.

13. Reference

[CSED 211] Malloc lab 문서

writeup문서

Computer Systems: A Programmer's Perspective(교과서)